Dual AI Personal Assistants - README

This project was developed to compare two AI assistants: a lightweight open-source assistant and a larger frontier model assistant. The system focuses on evaluating conversational quality, safety handling, memory support, hallucination behavior, and overall assistant performance.

Project Features

• Multi-turn conversations with short-term conversational memory

• Shared safety layer and regex guardrails for fair comparison

• Calculator and UTC time helper tools integrated into both assistants

• Evaluation framework using factual, jailbreak, and bias prompts

• JSONL logging for observability and debugging

Quick Start Setup

```
cd "gen ai"

python -m venv .venv

.venv\Scripts\activate

pip install -r requirements.txt
```

Environment Configuration

```
GROQ_API_KEY=gsk_your-key
GROQ_MODEL=llama-3.3-70b-versatile
OSS_MODEL_ID=Qwen/Qwen2.5-0.5B-Instruct
USE_HF_INFERENCE=false
```

OSS Backend Options

The project supports multiple OSS backend configurations:

• **groq** — Uses Llama 3.1 8B through Groq APIs for faster responses.

• **local** — Runs Qwen2.5 locally using transformers and PyTorch.

• **hf** — Uses Hugging Face inference APIs for supported models.

Running the Application

```
streamlit run app/streamlit_app.py
```

Evaluation

The project contains an evaluation framework for comparing both assistants. The benchmark includes factual prompts, adversarial prompts, and bias or safety-related prompts.

```
python scripts/run_eval.py
```

Project Architecture

```
app/streamlit_app.py

src/assistants/
  base.py
  oss_assistant.py
  frontier_assistant.py

src/memory/conversation_memory.py

src/guardrails/safety.py

src/tools/builtin_tools.py

src/eval/

src/observability/logger.py
```

Design Choices

• Shared guardrails and memory systems were used so both assistants could be compared fairly.

• Qwen2.5-0.5B was selected because it can run on CPU systems and also works well for free Hugging Face deployment.

• Streamlit was selected because it allows fast prototyping and simple UI development.

Tradeoffs

• Smaller OSS models are easier and cheaper to deploy but perform weaker on complex reasoning tasks.

• Regex-based safety systems are lightweight and fast but weaker against advanced jailbreak attacks.

• LLM-as-judge evaluation provides flexible scoring but increases API usage costs.

Future Improvements

• Add neural moderation and advanced guardrails

• Add RAG pipelines to reduce hallucinations

• Implement persistent vector database memory

• Add production deployment using FastAPI and authentication

• Add automated CI/CD evaluation pipelines

Bonus Features

The project also includes:

• Hugging Face Space deployment template
• Observability and logging support
• Cost and latency comparison notes

• Memory and tool integration

Security Notes

API keys and secrets should never be committed to GitHub repositories. The project includes a .gitignore file to prevent accidental uploads of .env files.

License

MIT License. External models remain subject to their respective licenses and policies.